Design layer

the whole layer, as one document

Contents

1 Sinal: typed native telemetry 5

1.1 Foundation 5
1.2 System at a glance 5
1.3 Effects and extension contracts 7
1.3.1 Effect timing and decision points 7
1.3.2 Caller codec and handler extension contract 7
1.3.3 Exporter and tracing adapter boundaries 8
1.3.4 Deployment, security and retention 8
1.4 End-to-end walkthrough 8
1.4.1 A correlated request with bounded observation 8
1.4.2 A scoped diagnostic and a native span 9

2 Forwarding 11

2.1 Foundation 11
2.2 Forwarder and route model 11
2.2.1 Forwarder incarnation and route lifetime 11
2.2.2 Public sums 12
2.3 Bounded forwarding and routing 13
2.3.1 Pure configuration and supervised startup 13
2.3.2 Admission and incarnation isolation 14
2.3.3 Ordering, shutdown and callback placement 15
2.3.4 Drop reports and accounting limits 15
2.3.5 Node-wide prefix routes 15

3 Observation 18

3.1 Foundation 18
3.2 Pending updates 19
3.3 Observation model 19
3.3.1 Concept and relationship census 19
3.3.2 Immutable definitions and values 21
3.3.3 Registration lifetime 22
3.3.4 Scope lifetime 23
3.3.5 Span invocation lifetime 24
3.3.6 Public sums and record boundaries 25
3.4 Definitions and codecs 25
3.4.1 Name admission and atom ownership 26
3.4.2 Native field representation 26
3.4.3 Record composition and effect timing 27
3.4.4 Enum completeness and non-silent incompatibility 27
3.5 Native dispatch and registration 28
3.5.1 Emission authority and inline execution 28
3.5.2 Pure subscriptions and installed identity 28
3.5.3 Decode failure, handler error and callback exception 29
3.5.4 Startup, detach and registry races 30
3.6 Subscription scopes 30
3.6.1 Acquisition, rollback and completion 31
3.6.2 Exceptional cleanup and resource limits 32
3.7 Correlation 32
3.7.1 Selection, validation and stable derivation 32
3.7.2 Shared metadata contract and propagation 33
3.8 Native spans 33
3.8.1 Event roles and owned fields 34
3.8.2 Execution, terminal selection and failure timing 34

4 Verification 37

4.1 Foundation 37
4.2 Validation profiles 37
4.3 Native oracle 38
4.3.1 Pinned source and mapping scope 38
4.3.2 Case mapping and dispositions 39
4.4 Focused checks and consumer obligations 39
4.4.1 Native and race harnesses 39
4.4.2 Consumers, examples and documentation 39

4.5 Benchmark method 40
Glossary 40

Sinal: typed native telemetry

1 Sinal: typed native telemetry

1.1 Foundation

Observations report facts and never decide them.

NO-GOAL

Observations do not control package state

Sinal does not decide authorization, retries, job progress, approval, workflow resume or compensation. Emitting a fact does not certify durable state or command success.

NO-GOAL

Export and trace parentage belong to adapters

External export, metric aggregation, unbounded buffering, trace propagation and handler recovery policy belong to applications or adapters. Sinal has no JSON Schema dependency or universal lifecycle event.

PRINCIPLE

Keep diagnostics separate from business authority

Correlations join observations rather than authorize work. Drop counts explain delivery limits rather than prove which effects occurred.

PRINCIPLE

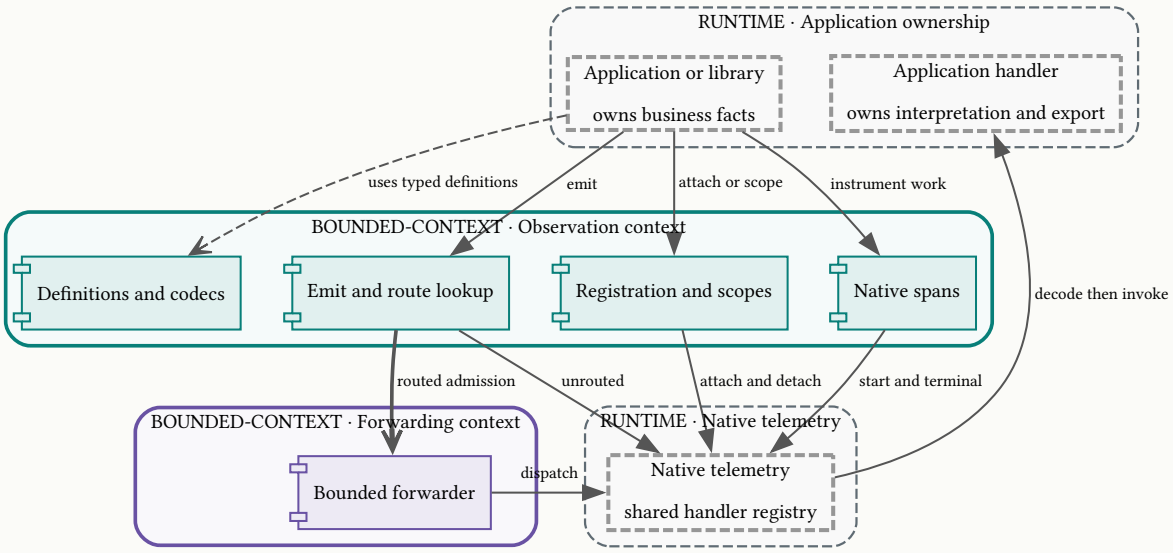
Make operational limits visible to callers

Process placement, resource retention, absence, refusal and cleanup remain observable contracts across every context. A convenient constructor does not imply delivery, quiescence or durability.

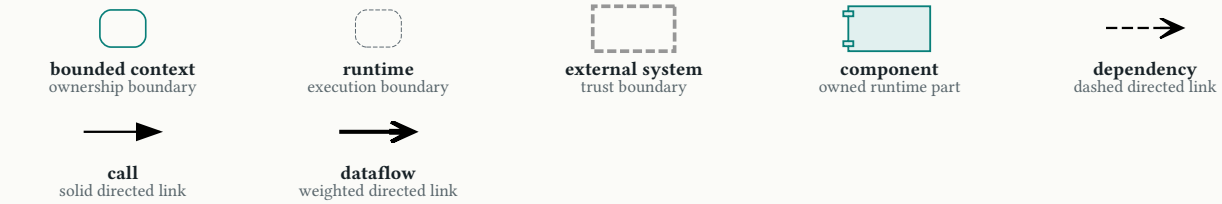
1.2 System at a glance

Three contexts divide typed observation, bounded forwarding and verification.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Observation ownership and execution



LEGEND



The boundaries mark each context's responsibility rather than a process. Unrouted callbacks share the producer process; forwarded callbacks share the forwarder process. Native spans and drop reports bypass routes.

- observation owns Event, Fields and Correlation. Its definitions allocate no event process and install no registration.
- Registration binds caller callbacks into Subscription values and returns Attachment handles. Scope orchestration owns ordered acquisition and reverse cleanup.
- Spans delegate timing and exception observation to native telemetry. forwarding owns optional process separation and admission through the Forwarder, while the Route table remains node-wide application configuration.
- verification owns the Native oracle, the Validation profile and the executable evidence for the other two contexts.
- All runtime work occurs on one BEAM node. There is no network transport, persistent event journal or cross-node registration; multiple nodes install independent handlers and routes.

Unit	Owns and composes with
Definitions and codecs (observation)	Names, trusted keys, caller types, foreign decoding and emit-time enum diagnostics.
Native dispatch and registrations (observation)	Handler identity, event selection, typed failures, native exceptions and detach.
Subscription scopes (observation)	Heterogeneous acquisition, rollback, work lifetime and indexed cleanup.
Correlation (observation)	Bounded shared observation value and optional or required field encoding.
Spans (observation)	Independent start and stop types, reserved timing fields and one catchable terminal observation.
Forwarding and routing (forwarding)	Incarnation admission, best-effort delivery, drop reporting and prefix selection.
Validation, oracle and benchmarks (verification)	Gate profiles, native parity, public consumer obligations, race probes and measurement method.

1.3 Effects and extension contracts

Every effect has a local owner; observation never changes the source package's decision.

1.3.1 Effect timing and decision points

Operation	Before authoritative effect	Effect and result boundary
event/fields/span definition	Validate source names, keys and role collisions; run pure placeholder plan	Allocate native atoms and immutable codec closures; no registration or event process.
subscription/plan	Bind typed callbacks and validate selected names	Retain closures only; no installed registry state.
attach	Select native identity; ensure telemetry application	Registry mutation; successful reply returns its handle. A failed reply can leave uncertain mutation.
emit	Check enums, encode both native maps, select route	Inline native dispatch or one admission/ send attempt; no delivery receipt.
with_subscriptions	Acquire complete known plan	Work runs, then cleanup; separate work and cleanup outcomes.
span.run	Encode start metadata	Start, work, completion encoding, terminal observation and return/re-raise in one process.
forwarder.new/supervised	Obtain stable name counters; validate startup settings	Publish incarnation target only after initialization; handlers execute later.
route/unroute	Validate trusted prefix	Serialize node-wide binding update; in-flight old delivery persists.

- The authoritative decisions are native registration success, selected codec admission, scope acquisition completion and forwarder admission. Native selection and dispatch do not confer business authority; a package emits only after its own decision point and keeps its result even if observation is refused.

1.3.2 Caller codec and handler extension contract

Caller-owned codec and callback integration

contract

Accept application values while retaining the shared native observation boundary.

responsibility

The caller defines key meaning, enum completeness, encoder/decoder laws and captured-resource lifetime.

interface

A custom field supplies a typed encoder and dynamic decoder; a typed handler supplies a fallible callback and failure observer before installation.

interactions

Sinal owns naming admission, map classification, event selection, installation and native failure semantics.

invariants

Caller extensions must not dynamically select keys, rely on decoder effects, change process placement or bypass the descriptor’s map boundary.

failure

Custom callback or codec exceptions remain native exceptions; typed handler errors retain caller e and native removal. Captured data can appear in native failure configuration.

- The extension contains no state type parameter beyond the caller’s measurement, metadata, result and error values. Application configuration remains in ordinary closures, and differently shaped subscriptions can be grouped after their types are bound.
- Names and identifiers serve different purposes: event names select observation classes; stable handler ids select registration ownership; correlations join work; SpanContext joins one native invocation. None is an authorization credential or universal idempotency key.
- Re-emitting an occurrence has no deduplication or replay guarantee. Duplicate stable attachment refuses, repeated detach reports absence, identical route writes replace, and deterministic correlation derivation preserves value equality without suppressing occurrences.

1.3.3 Exporter and tracing adapter boundaries

Observation export integration contract

Translate observations without moving business authority into the collector.

responsibility

An adapter owns external serialization, safe attributes, aggregation, export transport, retry and retention.

interface

It observes package-owned descriptors with ordinary typed handlers and supplies explicit process/buffering ownership.

interactions

Sinal supplies native telemetry interoperability, correlation values and optional bounded forwarding; packages keep their independent runtime decisions.

invariants

Absence, unknown measurements and native exception evidence are preserved according to an explicit adapter schema; observations do not resume or mutate work.

failure

Export errors cannot be treated as source-command failure or delivery confirmation; the adapter must bound its own memory and recovery policy.

- No exporter implementation, traceparent protocol, metric schema or durable queue is specified inside Sinal. Those retained integration capabilities require their own adapter contracts; the current package does not silently satisfy them through forwarding.
- Blueprint, workflow persistence, model providers, MCP, prompt management and retrieval are not runtime participants in this context. They may emit caller-owned facts using Sinal, but their schemas, permissions and state remain owned by those packages.

1.3.4 Deployment, security and retention

- Erlang/BEAM is the supported target; JavaScript is unsupported (ADR-0012) because the boundary uses native telemetry, maps, references, OTP processes, ETS, atomics and persistent_term. The package requires Gleam 1.18 or newer and supports telemetry 1.4.2 through later 1.x releases under the declared dependency range.
- Handler registry and routes belong to one node. Atom definitions and name-wide diagnostic counters can remain for its lifetime; attachment closure retention lasts until native removal, while queued event terms remain until dispatch or process death.
- No event payload size, native-map size, route-count, handler-count, capture-size or collector-history limit is enforced. Only correlation bytes, definition spelling and forwarder occurrence count have the explicit bounds described here.
- Security depends on source-trusted atom definitions and safe caller metadata. Native failure configuration exposes closures, and arbitrary exception terms can contain application details; exporters must select and redact safe fields under their own contract.
- Native telemetry's handler-table persistence is an in-memory runtime optimization rather than storage durability. There is no persistence adapter, transaction log or node-restart recovery layer to model; ADR-0013 leaves export and persistence to integrations.

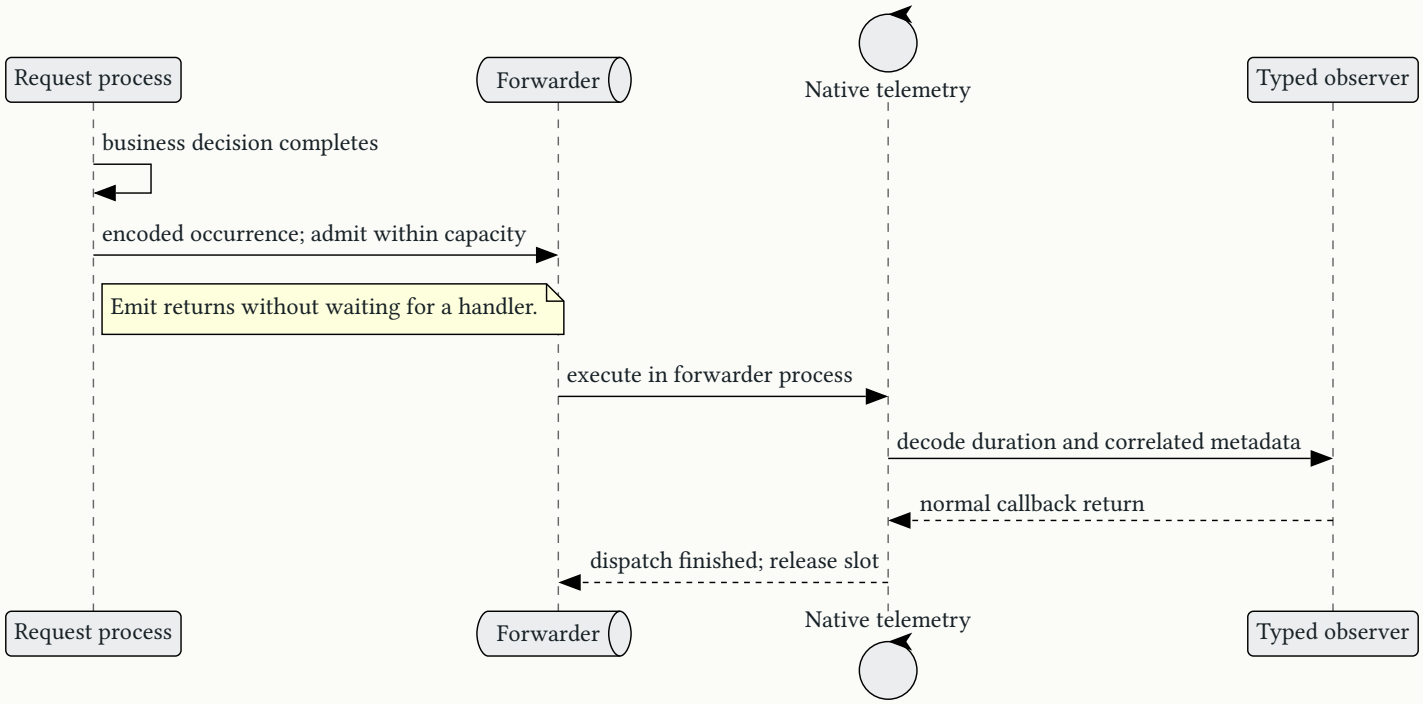
1.4 End-to-end walkthrough

One application's work identifier survives process separation without controlling the work.

1.4.1 A correlated request with bounded observation

- An application defines `[shop, request, finished]` with a duration measurement and metadata containing its route plus required correlation. It derives one correlation from its business request key and uses the named record pattern from Definitions and codecs.
- At application startup it installs a labelled typed observer, supervises one 1,024-capacity forwarder and routes `[shop]` to it. The observer sends safe records to the application's collector; it is installed once rather than once per request.
- The request code completes its own business operation and emits the resulting fact. Encoding and route selection happen in the producer, followed by one forwarder admission attempt; the business result remains the application's result regardless of refusal.

SEQUENCE Successful routed observation



The shared correlation is carried in metadata. The callback process is the forwarder and supplies no implicit request-local context.

- At capacity, the occurrence is dropped and counted as rejected. If the forwarder is unavailable it is counted as unavailable; a later running incarnation may emit the diagnostic report, but the application must not treat it as a per-request receipt.
- A malformed foreign occurrence of the same name notifies the failure observer and skips one callback. A handler-returned error removes the registration, while a process-level kill can stop the forwarder and cause a restart's best-effort lost report.
- Application shutdown removes its route and detaches the long-lived observer. Already admitted or selected callbacks can still finish, so collector resource release follows the application's own concurrency ownership rather than assuming detach establishes quiescence.

1.4.2 A scoped diagnostic and a native span

- A test or explicit diagnostic call creates an ordered plan for the span's typed start and stop observers. Scope acquisition completes before work starts, and those native span observations remain inline even while the shop prefix is routed.
- A returned business Error produces stop and remains the work result. A catchable throw produces exception with the start context and re-raises its original evidence; scope cleanup then attempts every known registration and reports indexed failures without replacing that outcome.
- These examples compose observation ownership rather than creating a shared runtime. The source package still chooses when its business state changed, the application still chooses retention/export and Sinal still provides only typed native observation plus explicitly selected bounded forwarding.

CONTEXT

Forwarding

2 Forwarding

2.1 Foundation

Applications choose process separation, and each running incarnation owns admission.

GOAL

Separate handler execution from the producer

An application can route event-name prefixes to a supervised forwarder process. A routed producer hands the occurrence over and continues without waiting for handlers.

GOAL

Bound admitted occurrences without blocking producers

A forwarder admits at most its capacity and refuses the rest immediately. Refusals are counted and reported as diagnostics.

NO-GOAL

Forwarding offers no delivery guarantee

A forwarder performs no acknowledgement, retry, drain, deadline or cancellation. Durable or confirmed export belongs to separate infrastructure.

INVARIANT

Admission never exceeds one incarnation's capacity

A forwarder's capacity includes queued and executing events. Its published destination and admission counter belong to the same running lifetime.

mechanism

INVARIANT

A refused routed occurrence never runs inline

A full or unpublished destination refuses the occurrence and counts it. The refusal never falls back to synchronous delivery in the producer.

mechanism

PRINCIPLE

Make operational limits visible to callers

Process placement, resource retention, absence, refusal and cleanup remain observable contracts. A convenient constructor does not imply delivery, quiescence or durability.

2.2 Forwarder and route model

A forwarder value survives restarts, while admission authority belongs to one running incarnation.

2.2.1 Forwarder incarnation and route lifetime

Admission authority belongs to the running incarnation. The application owns route configuration and supervision; a forwarder value survives an incarnation without carrying its admission rights.

Forwarder incarnation status initializing published stopped

Forwarder incarnation

aggregate · stateful · Typed telemetry · owned by Forwarding

One process lifetime with one published admission target.

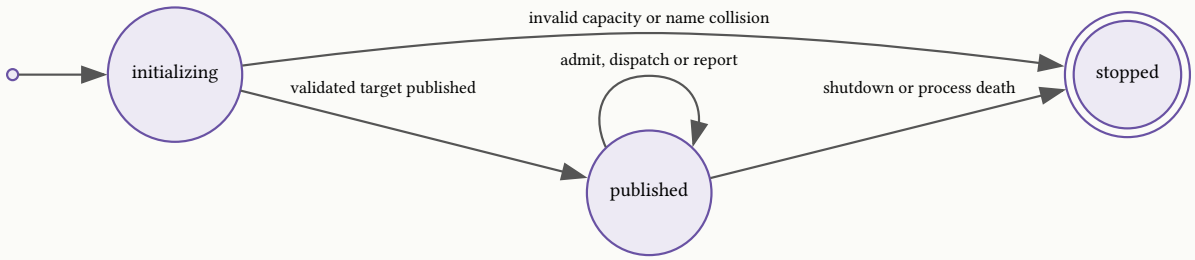
ATTRIBUTES

- authored **Name** · Application-owned node-local process name
  Stable across supervised restarts and reused by name-wide diagnostics.
- authored **Capacity** · Positive in-flight occurrence limit
  Validated before the target is published.
- derived **Target** · Direct subject and fresh admission counters
  Created at initialization and published as one incarnation-owned value.
- derived **Status** · Forwarder incarnation status incarnation-status · incarnation-lifecycle
  Publication authorizes admission; process death removes publication.

RELATIONSHIPS

- 1 : 0..n Owns accepted in-flight occurrences until dispatch finishes or the process dies.

STATE MACHINE Publication is the admission boundary



A restart creates a new machine instance. A delayed producer can send only to the direct subject it previously read, never to the replacement subject.

Route table

aggregate · stateful · Typed telemetry · owned by Application route configuration

The application's node-wide event-prefix bindings.

ATTRIBUTES

- authored **Bindings** · Unique prefixes with named forwarder destinations
  Route replaces exactly one prefix; unroute removes exactly one prefix.
- derived **Selection order** · Longest-prefix-first ordering
  Derived at each update; lookup selects the first matching prefix.

RELATIONSHIPS

- 1 : 0..n Names destinations regardless of whether a process is currently running.

- The route table has set membership and replacement operations rather than a single status field. Its transition contract is absent → bound, bound → replaced, and bound → absent for one exact prefix; unrelated bindings survive each operation.

2.2.2 Public sums

Refusal and drop reports classify delivery limits; the diagnostic counts are estimates rather than receipts.

Public type	Meaning and cases
Forwarder Refusal	CapacityExceeded; ForwarderUnavailable. Both mean the occurrence was refused rather than retried.

Public type	Meaning and cases
Dropped and DroppedMetadata	Rejected, lost and unavailable counts plus the forwarder's native name as text. Counts are diagnostic estimates.

2.3 Bounded forwarding and routing

Applications choose process separation while each running incarnation owns admission.

Forwarding contracts

responsibility

Run accepted occurrences in a separate named process without waiting for its handlers, and count best-effort refusal/loss diagnostics.

interface

`new`, `with_capacity`, `supervised`, `emit`, `route`, `unroute`, `dropped_event`, Dropped and Refusal.

interactions

OTP supervision owns process restarts; native telemetry still selects and runs handlers; node-wide route lookup selects an optional destination.

invariants

Queued plus executing occurrences never exceed one incarnation's capacity; old senders cannot target a replacement; diagnostic reports bypass routes.

failure

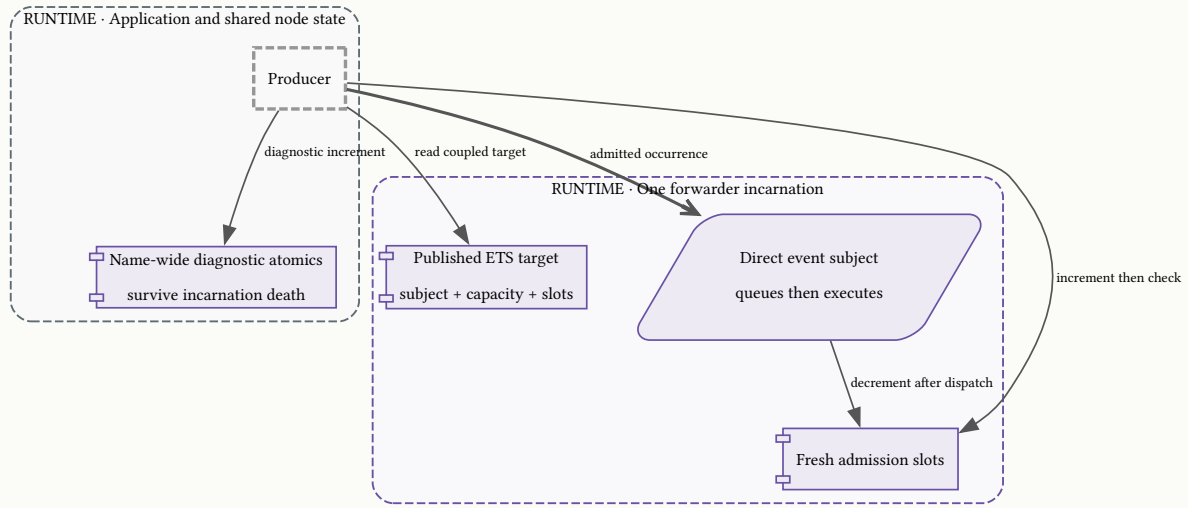
Full or unpublished targets refuse without wait/retry/fallback; process death loses pending work; report delivery is best effort; invalid capacity or target-name collision fails startup.

2.3.1 Pure configuration and supervised startup

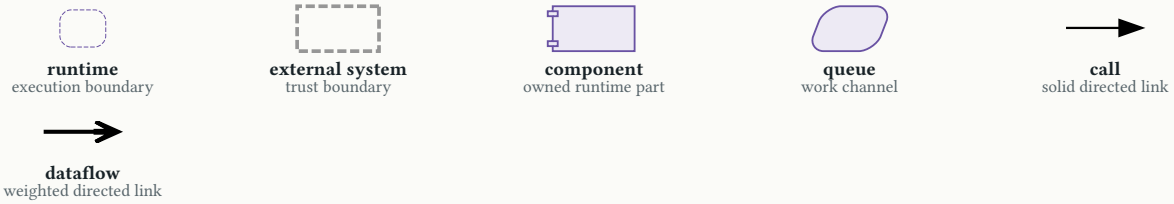
- `new(name)` creates no process and sets capacity to 1,024. It obtains name-wide diagnostic counters, so this constructor can allocate retained node-local counter state even though it does not start runtime execution.
- `with_capacity` changes the configured value. A capacity below one fails the supervised actor's initialization before publication; the initializer has a fixed 1,000 ms limit.
- `supervised` returns an OTP worker child specification whose startup result carries the Forwarder value. The application owns the supervisor, stable process name, shutdown policy and route registration.
- Startup creates fresh admission atomics and a direct event subject, drains name-wide diagnostics and publishes one target in a protected named ETS table owned by this incarnation. The ETS name is the same atom as the process name in the separate ETS namespace, so an existing table with that name fails initialization.
- User handlers run only after initialization from message handling. A producer before publication receives ForwarderUnavailable; startup never publishes a partially initialized admission target.
- One name creates retained shared diagnostic state for the node lifetime. Applications allocate names once at startup rather than per request; neither name count nor persistent_term memory has a Sinal quota.

2.3.2 Admission and incarnation isolation

VIEWPOINT RUNTIME · ALTITUDE L4 · INTERNALS Admission separates live slots from restart diagnostics



LEGEND



Only fresh incarnation slots authorize admission. Name-wide counts can be drained across restarts and never determine the replacement's capacity.

- A producer reads the direct subject, capacity and slots together. It atomically increments the local in-flight slot and refuses with CapacityExceeded if the result exceeds capacity, then rolls that increment back without sending the occurrence.
- Successful admission increments the separate name-wide in-flight diagnostic and sends directly to the captured subject. After native dispatch finishes, the incarnation decrements its local slot and floors the diagnostic decrement at zero.
- Increment-then-check may refuse a send that would fit under a different concurrent ordering. This safe spurious refusal is permitted near the boundary; admission never authorizes beyond capacity.
- The target table disappears with its owner process. A producer delayed after reading an old target can send only to that old direct subject and modify only its old slots; it cannot enqueue into the replacement incarnation.
- Ok(Nil) from direct forwarder.emit means admission and send, not completion or durable receipt. A process can die after lookup or admission; accepted occurrences may still be lost, and the restart's diagnostic estimate can undercount or overcount racing work.
- Capacity bounds event count, not message bytes, encoder allocations, captured handler data or downstream collector buffers. Values are encoded in the producer before admission, so even a refused oversized value has already been constructed and encoded.

BEHAVIOR

A full destination refuses without delaying its producer

Given the selected forwarder has no available admission capacity

When a producer hands it an occurrence

Then the occurrence is refused and counted as rejected

Then the producer does not wait for a handler or retry the occurrence

level: boundary area: Bounded forwarding and routing

BEHAVIOR

An unpublished destination refuses as unavailable

Given no initialized target is published for the named forwarder

When a producer attempts to emit to that destination

Then the occurrence is refused and counted as unavailable

Then the producer does not fall back to inline dispatch

level: boundary area: Bounded forwarding and routing

- ADR-0008 records why incarnation-local authority is separate from diagnostics. The delayed-start, delayed-sender and delayed-drop probes synchronize the exact races instead of using elapsed sleeps as evidence.

2.3.3 Ordering, shutdown and callback placement

- Admitted events from one producer to the same destination execute FIFO. Interleaving across producers is unspecified, and events split across routes or route changes have no relative-order guarantee.
- Native handlers run in the forwarder process, serially for forwarded occurrences. A blocking handler blocks the forwarder, consumes one executing slot and causes later producers to fill or exceed capacity without blocking those producers.
- Catchable callback failures retain native removal and allow the forwarder to continue. A handler linked to a failing process, or an untrappable kill, can still terminate the forwarder; isolation moves that execution risk to a supervised process rather than making callbacks harmless.
- Shutdown does not drain the queue or wait for effects triggered by handlers. Pending work is lost, and restart snapshots leftover name-wide in-flight count as lost before starting fresh admission counters.
- Forwarding offers no acknowledgement, retry, deadline or cancellation operation. Applications use separate delivery infrastructure when durable or confirmed export is required.

2.3.4 Drop reports and accounting limits

- The native event name is `[sinal, forwarder, dropped]`. Its integer measurements are rejected, lost and unavailable; metadata carries the forwarder name under forwarder.
- Rejected counts capacity refusals, unavailable counts absent targets, and lost estimates admitted work remaining from the previous incarnation. All three diagnostic slots are shared by every Forwarder value built from one name.
- Startup drains all three counts and queues a report only when any is nonzero. A running drop drain exchanges rejected and unavailable to zero and reports them with lost set to zero.
- A local per-incarnation flag coalesces drop notifications into one pending notice. It is cleared before draining so a racing refusal can schedule the next notice; a delayed old notification cannot target the replacement or change its flag.
- Reports execute native telemetry directly in the forwarder and bypass every route, including the empty prefix. A slow report observer can block the forwarder, but report emission cannot recursively create a forwarded drop.
- Drained counts are not restored if the process dies before reporting them. If the forwarder never starts again, unavailable drops may remain unreported for the rest of the node lifetime; direct Refusal is the only immediate signal to an explicit caller.

BEHAVIOR

Drop reporting cannot recurse through a route

Given a drop drain contains a nonzero diagnostic count

When the running incarnation reports that drain

Then the report bypasses route selection and exposes exactly the drained counts

Then reporting the drop does not itself require forwarder admission

level: boundary area: Bounded forwarding and routing

2.3.5 Node-wide prefix routes

- route binds a trusted prefix to a forwarding closure in node-wide persistent_term. Empty prefix matches every ordinary event; longest prefix wins, and writing the same prefix replaces its destination.
- Route writers serialize through a node-local lock and publish one sorted route-list value. Readers perform one persistent_term lookup and scan longest-first without acquiring that lock; route count and scan length are unbounded.
- Unroute removes exactly one prefix. Later events select a shorter matching route or execute inline, while events already handed to the old forwarder may finish after those later inline events.

- Persistent-term replacement or removal can impose node-wide runtime cost. Route changes belong to application startup/shutdown or rare explicit reconfiguration rather than each request.
- `sinal.emit` returns Nil and discards a selected forwarder's typed refusal after recording it. `forwarder.emit` ignores routes and returns Refusal directly; spans and drop reports always remain native inline operations.
- No default forwarder application starts itself. An application supervises its chosen process before routing, then removes its route as part of shutdown; a route to an unavailable process keeps dropping rather than silently changing execution placement.

BEHAVIOR

Longest-prefix routing selects one destination

Given several configured route prefixes match an emitted name

When ordinary emission resolves the route

Then the longest matching prefix selects the forwarder

Then one selected destination receives the admission attempt

level: boundary area: Bounded forwarding and routing

- ADR-0003 records the ordinary route-aware path and refusal policy. The SSO Portal consumer supervises a forwarder and routes HTTP Gun's prefix once; Grind independently owns runtime-local forwarders and can retain its direct refusal boundary.

IMPLEMENTATION AND OPERATIONAL EVIDENCE

Forwarder, incarnation publication and atomics, route port, capacity/restart tests, route tests and barrier race probes own these guarantees.

CONTEXT

Observation

3 Observation

3.1 Foundation

Typed definitions, registrations, scopes, correlations and spans share the native BEAM contract.

GOAL

Emit and observe caller-owned Gleam values

One event definition binds a native name to independent measurement and metadata codecs. Producers and observers reuse it without erasing their application records.

GOAL

Interoperate with native telemetry producers and handlers

Erlang and Elixir code receives atom-list names and native maps. Foreign maps are decoded before typed handlers run.

GOAL

Provide explicit observation lifetimes

Applications own long-lived attachments, scoped groups and native spans. Each lifetime states what it retains and what it releases.

INVARIANT

Typed handlers receive decoded declared values

Both maps must decode through the selected event before its typed callback runs. A malformed map is never passed as a partially decoded application value.

mechanism

INVARIANT

Work exceptions retain their original identity

A catchable work error, exit or throw retains class, reason and stacktrace through scoped cleanup and native span execution. Cleanup reporting cannot replace a work exception.

mechanism

INVARIANT

Atom-producing definitions come from trusted source

Event names, route prefixes and field keys must be source definitions (see Source definition). Grammar checks reject invalid spelling but cannot prove trust or bound the number of valid names.

convention

PRINCIPLE

Preserve native ownership at the observation boundary

The native registry owns handler selection and inline execution. Sinal supplies typed definitions and explicit application-selected isolation.

3.2 Pending updates

ruling **Recover an attachment after an uncertain native call** 2026-10-06

Native registry calls can raise or time out after their result becomes uncertain. A scope can clean up only registrations whose acquisition returned a handle, and automatic identifiers stay hidden from the caller when acquisition raises. No generation token, cancellation protocol or uncertain-acquisition recovery contract is specified.

ruling **Trace parentage and propagation requirements** 2026-10-06

A correlation can carry a trace identifier, but neither it nor a native span context carries parent-child semantics. Causation fields and OpenTelemetry propagation remain requirements to settle when a tracing adapter is designed.

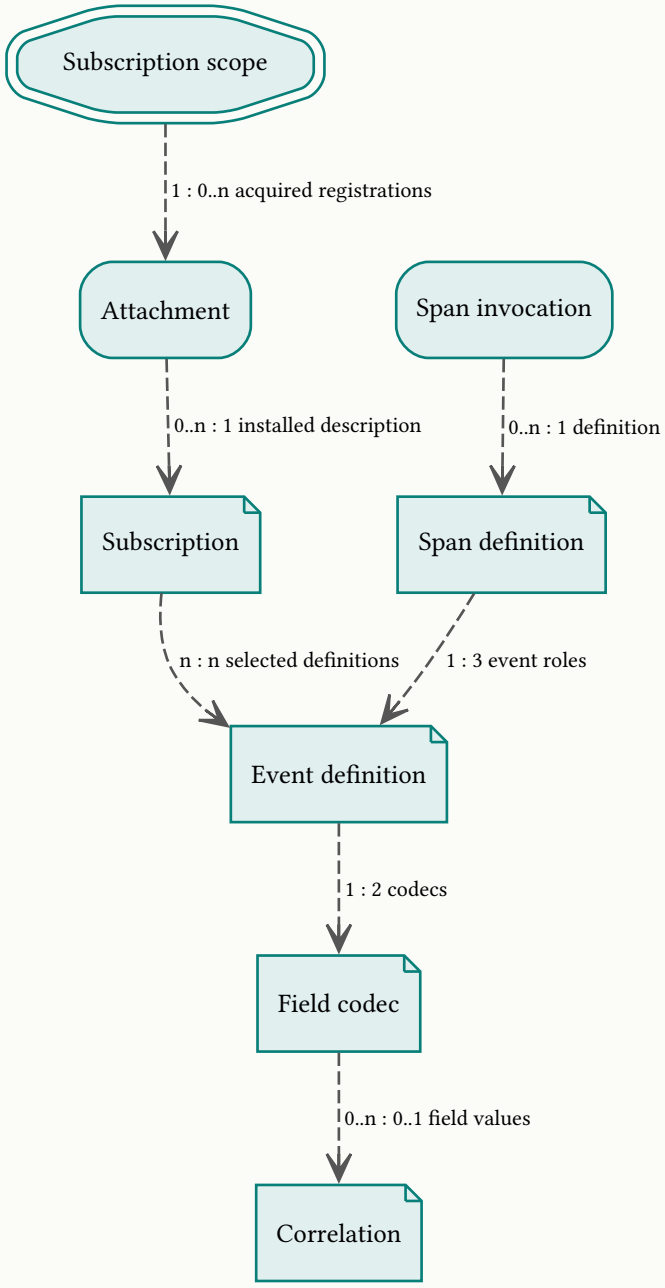
ADR-0007

3.3 Observation model

Definitions, registrations and recorded occurrences have distinct ownership.

3.3.1 Concept and relationship census

Eight model subjects describe its values and live ownership: event definition, field codec, correlation, subscription, attachment, scope, span definition and span invocation. The forwarding context owns the forwarder incarnation and route table. Lifecycle states describe observable control flow rather than requiring those exact runtime records.



LEGEND

value object
attribute-defined value

entity
identity-bearing record

aggregate
invariant-owning cluster

dependency
dashed directed link

Edges describe typed definitions and ownership, rather than delivery order. A descriptor sharing a raw name with another descriptor does not establish global schema agreement.

3.3.2 Immutable definitions and values

Event definition

value-object · immutable · Typed telemetry · owned by Definitions

A reusable name and two codecs.

ATTRIBUTES

- authored **Name** · Nonempty trusted event name
  Source-defined segments satisfying the name grammar.
- authored **Measurement shape** · Measurement field codec
  The caller measurement value and its native keys.
- authored **Metadata shape** · Metadata field codec
  The caller metadata value and its native keys.

RELATIONSHIPS

- 1 : n May be selected by many subscriptions.

Field codec

value-object · immutable · Typed telemetry · owned by Definitions

One declared native-map shape and caller value type.

ATTRIBUTES

- derived **Declared keys** · Ordered distinct trusted keys
  Derived at definition from the field-composition plan.
- authored **Encoding and decoding** · Caller-value transformations
  Getters and decoders preserve the declared meaning of each field.

RELATIONSHIPS

- n : n Composes reusable field groups into event and span roles.

Correlation

value-object · immutable · Typed telemetry · owned by Correlation

One bounded observation identifier chosen by an application.

ATTRIBUTES

- authored **Value** · UTF-8 text of one to 128 bytes
  Provided verbatim, derived from a key, or randomly generated before constructing the value.

RELATIONSHIPS

- 1 : n May join many work-scoped observations.

Subscription

value-object · immutable · Typed telemetry · owned by Registration

A typed callback bound to one or more same-shaped event definitions.

ATTRIBUTES

- authored **Selected events** · Nonempty distinct event-name set
  Every selected definition has the same measurement and metadata type parameters.
- authored **Callbacks** · Handler and typed failure observer
  Closures can capture independent caller-owned configuration.

authored **Identity preference** · Fresh identity or nonempty stable identity
A readable label may decorate fresh identity; stable identity overrides the label.

RELATIONSHIPS

1 : n May be installed repeatedly as distinct fresh attachments.

Span definition

value-object immutable Typed telemetry owned by Spans

The three typed event roles surrounding instrumented work.

ATTRIBUTES

authored **Name prefix** · Nonempty trusted event name
The start, stop and exception role suffixes extend this prefix.

authored **Caller shapes** · Start metadata, extra stop measurements and stop metadata
Independent types prevent an implicit transfer of start fields into stop metadata.

RELATIONSHIPS

1 : n Defines arbitrarily many independent invocations.

3.3.3 Registration lifetime

Native registration owns this lifecycle. An attachment's handle does not mutate when native telemetry removes its registration; its status is observed by lookup and detach.

Registration status

vacant attached detached failed

Attachment

entity stateful Typed telemetry owned by Native registry

A handle to one native handler identifier.

ATTRIBUTES

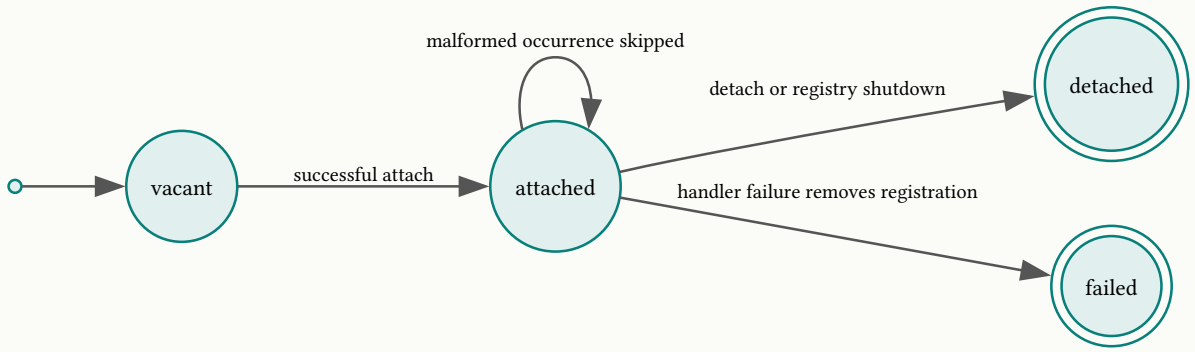
derived **Identifier** · Node-local native handler identifier
Derived at successful installation, or taken from the caller's stable identity.

observed **Status** · Registration status attachment-status · attachment-lifecycle
The current registry relationship; it is not cached authority inside the handle.

RELATIONSHIPS

n : 1 Installs one subscription description.

STATE MACHINE Native registration transitions



A duplicate stable identifier refuses a new acquisition and leaves the existing registration unchanged. Reusing an identifier creates a new registration; a stale handle can detach that replacement.

3.3.4 Scope lifetime

A scope owns only attachments whose acquisitions completed successfully. A typed refusal skips work; a catchable exception leaves through cleanup and re-raise.

Scope status acquiring working cleaning completed refused raised

Subscription scope aggregate stateful Typed telemetry owned by Scope orchestration

One ordered acquisition and cleanup operation.

ATTRIBUTES

authored **Plan** · Ordered subscription plan
Includes the exceptional cleanup reporter.

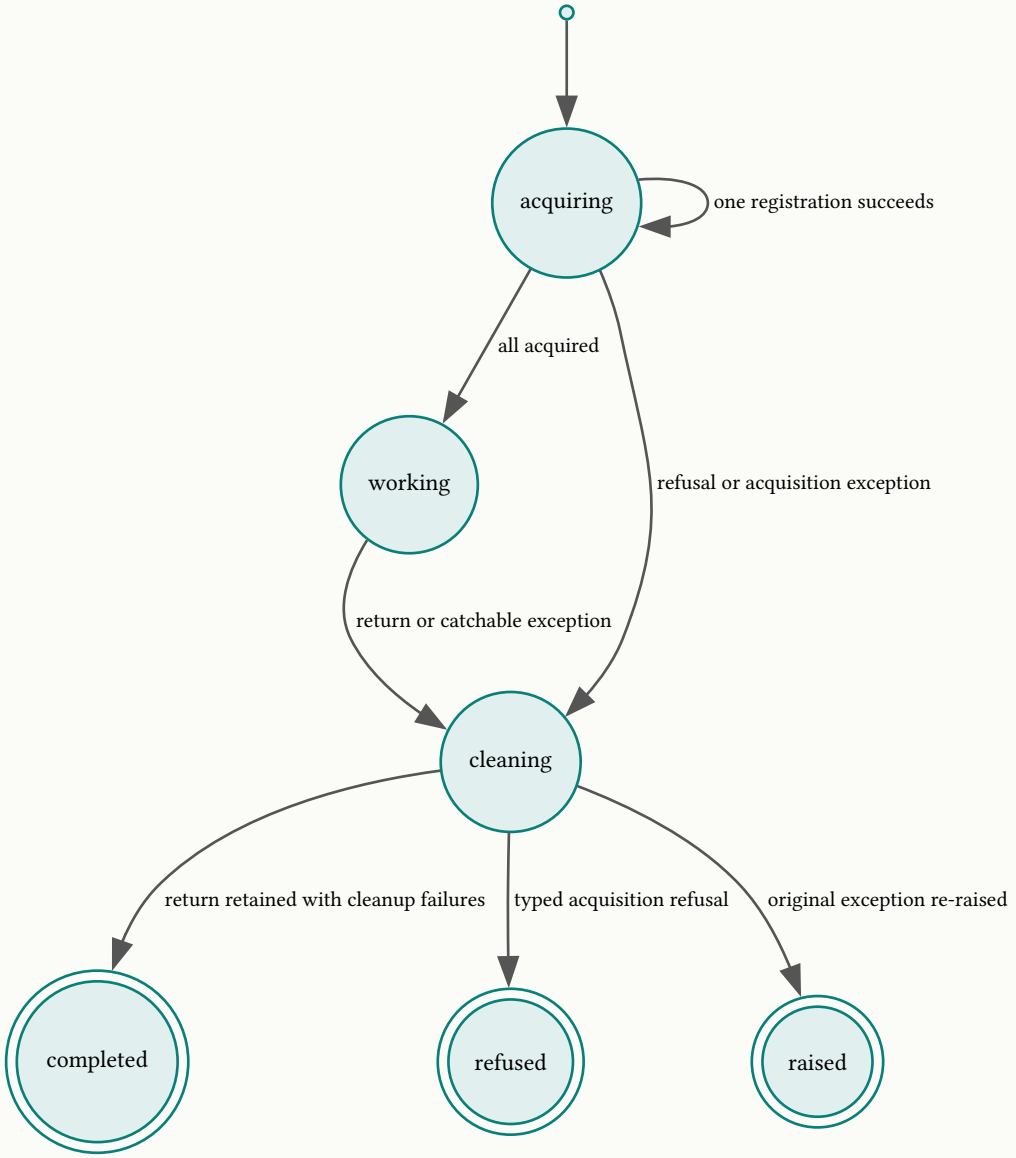
derived **Acquired registrations** · Indexed successful attachments
Derived at acquisition and stored newest first for reverse cleanup.

derived **Status** · Scope status scope-status · scope-lifecycle
Derived from the current acquisition, work or cleanup step.

RELATIONSHIPS

1 : 0..n Owns acquired attachments until cleanup is attempted.

STATE MACHINE Scoped acquisition and cleanup



Cleanup outcome does not replace the work outcome. An untrappable kill, node loss or VM loss can interrupt any nonterminal state without following a cleanup transition.

3.3.5 Span invocation lifetime

One native context identifies each invocation. The terminal choice is the work's normal return or catchable exception, independent of observation callback success.

Span invocation status not-started running stopped exception

Span invocation entity stateful Typed telemetry owned by Native span runtime

One native execution of a span definition.

ATTRIBUTES

derived **Context** · Native span context
Created at invocation and shared by its start and terminal occurrences.

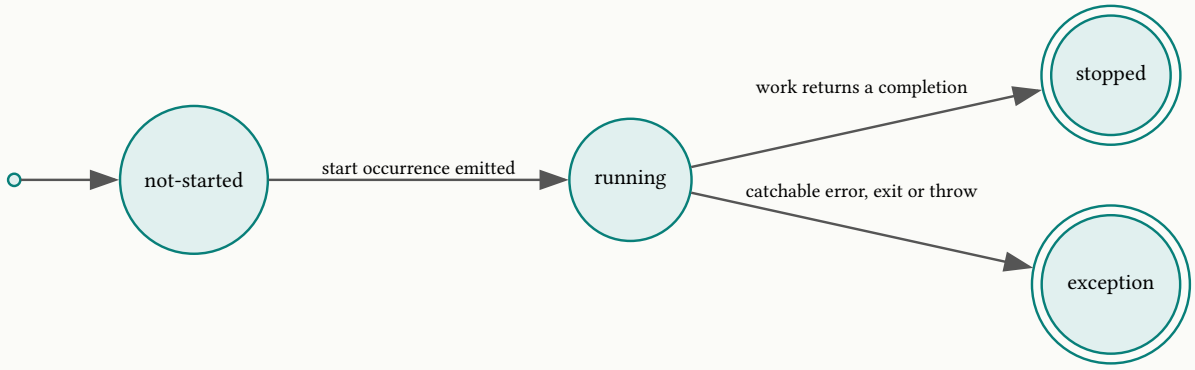
derived **Status** · Span invocation status span-status · span-lifecycle
Derived from native execution flow.

observed **Timing** · System time, monotonic time and native duration
Observed from the BEAM runtime; duration is a difference of monotonic readings.

RELATIONSHIPS

n : 1 Runs one span definition.

STATE MACHINE Normal return and exception choose different terminals



A returned application Error is normal completion. Uncatchable process termination can prevent any terminal occurrence.

3.3.6 Public sums and record boundaries

These sums classify observable failure; diagnostic strings are descriptions rather than machine-readable decisions. Public returned records are read by label, so callers must not infer a frozen positional representation.

Public type	Meaning and cases
FieldError	NotAMap; MissingField(key); InvalidField(key, decoder errors). First failure follows declaration order.
AttachError	AlreadyExists(id). Unexpected native availability, startup or call exceptions are not folded into this sum.
HandlerFailure(e)	MalformedMeasurements; MalformedMetadata; HandlerReturned(e). Error values remain caller-owned.
CleanupFailure	AlreadyDetached; DetachCrashed(description). Scope cleanup retains the failing zero-based plan index.
SubscriptionScopeError	SubscriptionAttachFailed(index, attach error, rollback failures). Work never ran.
SubscriptionCompletion(a)	The normal work value and reverse-cleanup failure list. A value of Result.Error is still a normal work value.
CorrelationError	EmptyCorrelation; CorrelationTooLong(actual bytes, maximum bytes).
Span Completion	The work result, extra stop measurements and independent stop metadata.
Span event records	Start/stop/exception measurements and metadata retain separate role types; native time wrappers and context stay opaque.
ExceptionKind and TimeUnit	Error/Exit/Throw classify native exceptions; Native/Nanosecond/ Microsecond/Millisecond/Second select conversion units.

- There is no stored business aggregate, journaled command, persisted lifecycle or transactional event stream. Observations are transient occurrences; the native registry is node-local runtime state.

3.4 Definitions and codecs

Caller types cross the native-map boundary without compulsory serialization.

Definition and codec contracts

responsibility

Bind trusted event identity and field meaning to caller-owned values.

interface

`event`, name/codecs accessors, primitive `Fields`, `include`, `success`, `optional`, `enum`, `field`, `encode`, `decode`, `check` and `keys`.

interactions

Native dispatch, spans and direct forwarding share the same codecs; foreign callbacks always decode.

invariants

Names are nonempty; grammar-valid declared keys are unique in one map; separate measurement and metadata maps can reuse a key.

failure

Source-definition defects panic at construction; malformed maps return FieldError; caller codec exceptions remain BEAM exceptions.

3.4.1 Name admission and atom ownership

- Each event-name segment and field key matches `[a-z][a-z0-9_]{0,62}` before atom creation at definition. Event and span names are nonempty; an empty route prefix is permitted because it intentionally selects every event.
- Validation is per segment, with a maximum of 63 ASCII bytes. There is no name-segment-count limit, key-count limit, descriptor-count limit or node atom-count quota.
- A valid dynamically generated tenant name can still create unbounded permanent atoms. Applications must keep variability in metadata values rather than event names, field keys or route prefixes; ADR-0002 records the source-definition boundary.

BEHAVIOR

Invalid source definitions fail before use

Given a definition has an invalid or empty required name, duplicate declared key, invalid optional shape, ambiguous enum name, reserved span key or duplicate selected event

When the caller constructs the definition

Then construction raises a programmer-error diagnostic naming the offending definition

Then the definition cannot be used to install a handler or emit an occurrence

level: boundary area: Definitions and codecs

3.4.2 Native field representation

Codec	Encoded value	Decode contract
`string`	UTF-8 binary	Accept the Gleam string decoder's binary representation.
`int`	Integer	Accept an integer.
`float`	Float	Accept a float or an integer converted to float.
`bool`	Boolean atom	Accept a boolean.
`enum`	The caller naming function's UTF-8 binary	Accept that binary or an existing atom whose text matches a declared value name.
`field`	Any caller-encoded BEAM term	Run the caller's dynamic decoder at one declared key.
`optional`	None omits its only key; Some writes the inner value	Absent key, nil or undefined means None; every other term uses the inner decoder.
`empty` or `success`	Empty map	Require a map and return the fixed success value while ignoring unrelated keys.

- Native map keys are atoms, not binary aliases. Extra foreign keys are ignored, while a missing required key and an invalid present value remain distinct errors.
- `optional` requires exactly one declared key. Its inner encoding must not use native `nil` or `undefined` as meaningful present values because those markers denote absence.

- `fields.encode` runs the declared getters and encoders against an empty native map. Its type has no operational error return; this does not protect the caller from a crashing custom getter, encoder or decoder.

BEHAVIOR

Foreign values are checked at every typed boundary

When a caller decodes a foreign occurrence's declared map shape

Then unrelated keys do not alter the declared value

Then a non-map, absent required field or invalid present field returns a distinct classified failure

level: boundary area: Definitions and codecs

3.4.3 Record composition and effect timing

- `include(field, then:, get:)` combines the field's keys with the remaining record's keys in declaration order. The getter extracts the encoded value, while the bound decoded value enters the labelled record constructor in `success`.
- A nested record codec flattens its declared keys into the same native map. A duplicate across any flattened group panics at definition, while declaration order determines the first reported decode or enum-check failure.
- The encoding plan is built once by running the continuation with placeholder values. `field` obtains its placeholder by running the custom decoder against `nil` while discarding decode errors; therefore the decoder and builder can execute at definition before any observation exists.
- Decoding reruns the continuation with actual decoded values. The process-local decoding marker prevents eager reconstruction of encoding plans during that pass and is restored after either return or exception; nested decoding preserves the surrounding marker.
- Field selection must not depend on a prior bound value. Such selection could give the definition-time encoder and occurrence-time decoder different shapes; the public builder is intended to contain `include` calls and one `success` constructor with pure getters and decoders.

CODE GLEAM

```
pub type Delivery { Delivery(route: String, status: Int) }

pub fn delivery_fields() -> fields.Fields(Delivery) {
  use status <- fields.include(fields.int("status"), get: fn(m) { m.status })
  use route <- fields.include(fields.string("route"), get: fn(m) { m.route })
  fields.success(Delivery(route:, status:))
}
```

- Here constructor labels preserve the route/status relationship despite the opposite declaration order. Named binding prevents a silent swap of two same-typed fields; ADR-0006 records the choice.

3.4.4 Enum completeness and non-silent incompatibility

- `enum` requires a nonempty value list with unique encoded names. An exhaustive naming function does not prove that the list includes every constructor of the caller's type.
- Encoding an unlisted value still writes its name for native observers. `fields.check` returns InvalidField for that value, while the public emit paths log the same incompatibility in the producer process and still emit the occurrence.
- A typed observer's enum decoder rejects the unlisted name and skips that invocation. This is a definition-completeness risk rather than a delivery receipt or a handler-returned application error.

BEHAVIOR

An unlisted enum value remains diagnosable

Given a caller value is absent from its enum's declared value list

When the caller emits that value

Then the encoded name remains available to native observers

Then the producer receives an incompatibility warning naming the event and field

Then a typed observer reports malformed input and remains installed if its failure observer returns normally

level: boundary area: Definitions and codecs

- Applications test every named constructor with `fields.check` and encode/decode roundtrips. Event measurement and metadata accessors expose codecs so these checks and exact native-map assertions require no registration; this obligation is not enforced by the compiler.

IMPLEMENTATION AND BEHAVIORAL EVIDENCE

Fields, name admission, emit diagnostics, codec tests and enum completeness tests own the current executable boundary.

3.5 Native dispatch and registration

The native registry selects callbacks; Sinal selects and decodes their typed definition.

Native dispatch contracts

responsibility

Translate typed values to native maps and bind selected definitions to installed callbacks.

interface

`emit`, `observe`, `observe_labelled`, `subscription`, `handler`, `with_label`, `with_id`, `attach`, `detach` and failure descriptions.

interactions

The native telemetry registry runs exported sinal_ffi:handle/4 with the typed closure as configuration. Routes optionally change the dispatch process.

invariants

Selected event names are distinct inside one handler; one native identifier names one registration at a time; measurement decoding precedes metadata decoding and handler invocation.

failure

Malformed input skips one invocation; returned handler error or callback exception removes the whole registration; native call failures can raise beyond AttachError.

3.5.1 Emission authority and inline execution

- `sinal.emit` encodes measurements and metadata before looking up the longest matching route. With no route, native execute invokes selected handlers in the current producer process before returning Nil; ADR-0001 records native ownership.
- A slow or blocked handler delays the producer without a Sinal deadline. Different producer processes may concurrently invoke the same handler, and the native registry supplies no order between handlers selected for one occurrence.
- The callback sees the native execution process as its `self()`. Process-dictionary state is whatever exists in that process; forwarding does not copy producer-local context, so work identity must travel as an explicit metadata field.
- Handler failure isolation covers catchable callback error, exit and throw. Arbitrary linked-process exits, untrappable kill, custom encoder exceptions or VM termination can still terminate the current process; a callback's failure isolation is not a guarantee against every action a handler can perform.

BEHAVIOR

Unrouted dispatch completes in the producer

Given the occurrence's name matches no route

When a producer emits the occurrence

Then selected handlers execute in the producer's process before emission returns

Then a catchable handler failure does not become the producer's application error

level: boundary area: Native dispatch and registration

- A descriptor constrains cooperating Gleam callers, not the node's raw name. Independent descriptors can share the same name with different codecs, and every typed callback therefore decodes rather than trusting the producer's type.

3.5.2 Pure subscriptions and installed identity

- `observe` is the common infallible one-event path with a fresh identifier and warning logger for decode failures. `observe_labelled` adds a readable label without requiring label uniqueness or a typed attach result.

- `subscription` describes the same infallible callback without installing it. `handler` describes one fallible callback and failure observer for a nonempty list of same-shaped events; it returns the selected descriptor to both callbacks and refuses repeated names as definition bugs.
- Different measurement or metadata shapes belong in separate subscriptions. Their callbacks remain typed before the opaque Subscription values are grouped into a heterogeneous plan.
- Fresh native identity is `{sinal_handler, N}` or `{sinal_handler, Label, N}`, where N comes from a node-local positive unique integer. Labels accept arbitrary strings, create no atoms, and are diagnostic names rather than collision keys.
- `with_id` replaces the complete native identity with the supplied nonempty string. A duplicate returns AlreadyExists(id); a stable identity can be detached by raw native code, and labels do not decorate that stable identity.

BEHAVIOR

A duplicate stable identifier preserves the incumbent

Given the subscription's chosen identifier is already in use

When the caller attaches the subscription

Then attachment is refused with the conflicting identifier

Then the existing registration is unchanged

level: boundary area: Native dispatch and registration

- Subscription construction installs nothing, but opaque closures may retain caller resources. Applications bound the count and lifetime of subscriptions and captured values; no library subscription-count or capture-size limit exists.

3.5.3 Decode failure, handler error and callback exception

Outcome	Typed observer	Native consequence
Measurements fail to decode	MalformedMeasurements; metadata decode and run are skipped	Registration stays if failure observer returns.
Metadata fails to decode	MalformedMetadata; run is skipped	Registration stays if failure observer returns.
Run returns Ok	No failure notification	Registration remains.
Run returns Error(e)	HandlerReturned(e), then adapter raises	Entire registration removed; native failure occurrence emitted.
Run or failure observer raises	Unexpected exception does not enter the caller's e	Entire registration removed; native failure occurrence emitted.

- The ordinary subscription logs a decode warning and retains the registration. A custom failure observer that raises converts even a malformed-data path into native callback failure; ADR-0004 records the retained-handler rule.
- A multi-event registration fails as one identity. Failure during one selected event removes it from every selected event, while an unrelated registration continues to receive occurrences.
- Native `[telemetry, handler, failure]` metadata includes the triggering name, native handler identifier, callback configuration, exception class, reason and stacktrace. The configuration is the actual typed callback closure; captured application values are visible to native failure observers and handler-table introspection.
- Opaque Subscription and Attachment types do not make captured data secret from native code. Applications must avoid capturing credentials or other sensitive data they cannot expose to installed native observers; Sinal does not redact that native configuration.

BEHAVIOR

Malformed data skips one callback invocation

Given one selected occurrence has malformed measurements or metadata

Given the failure observer returns normally

When the native registration receives the occurrence

Then the typed work callback does not run for that occurrence

Then the failure observer receives the classified decode failure and selected event

Then later compatible occurrences can still reach the registration

level: boundary area: Native dispatch and registration

BEHAVIOR

A failed handler removes its whole registration

Given a selected typed handler returns an error or its callback raises

When native dispatch handles that failure

Then the registration is removed from every selected event

Then native failure observers receive the identifier, configuration and exception evidence

level: boundary area: Native dispatch and registration

3.5.4 Startup, detach and registry races

- Attach checks the native handler-table process and starts the telemetry application through ensure_all_started when absent. Application-start failure raises a BEAM error; it is not AlreadyExists, and no standalone Sinal registry is created.
- Emit does not start telemetry. With no running native registry it reaches no handler; detach returns Error(Nil) when the registry or identifier is absent.
- Native attach and detach use the registry's ordinary synchronous call timeout, documented as 5,000 ms for the oracle. Registry death or a call timeout can raise, and a call whose reply is not received may have an uncertain mutation outcome.
- Detach removes future selection by identifier but does not wait for an already selected callback. A second detach reports absence rather than idempotent success.
- Handles are copyable and native removal has no compare-and-delete generation token. If a stable identity or deliberately inspected fresh identity is reused after removal, a stale handle can detach its replacement.
- Native `telemetry.persist/0` is available to Erlang/Elixir or a caller FFI and preserves registrations across its storage migration. Sinal intentionally exposes no additional persist wrapper or private introspection registry.

BEHAVIOR

Detach leaves selected callbacks able to finish

Given another process already selected the callback

When the caller detaches its registration

Then later lookup cannot select that registration

Then the selected callback may finish after detach returns

level: boundary area: Native dispatch and registration

IMPLEMENTATION AND BEHAVIORAL EVIDENCE

Public dispatch and registration, native registry boundary, native interoperability and failure tests and fresh-VM startup test describe this boundary without claiming unknown-call recovery.

3.6 Subscription scopes

A scope retains work outcomes while making registration cleanup visible.

Scope ownership contracts

responsibility

Acquire an ordered plan, run work only after complete acquisition, and attempt reverse cleanup.

interface

`subscriptions`, `with_exception_cleanup_reporter`, `with_subscriptions`, indexed failure records and completion/error descriptions.

interactions

Native attach/detach supply effects; the Erlang scope shim captures all catchable exception classes and preserves re-raise evidence.

invariants

Every known successful acquisition receives a cleanup attempt on ordinary or catchable exit; cleanup never replaces the normal work value or original exception.

failure

Typed acquisition refusal returns an indexed error plus rollback failures; acquisition/work exception re-raises after cleaning known registrations; untrappable termination can skip cleanup.

3.6.1 Acquisition, rollback and completion

- A SubscriptionPlan may be empty and combines subscriptions with unrelated caller measurement, metadata and error types. Its exceptional cleanup reporter defaults to a silent callback.
- Acquisition follows list order with zero-based indexes. Each successful attach adds a cleanup closure to the front of the acquired list, fixing reverse cleanup order without a later sort.
- A typed refusal detaches all preceding successful acquisitions and returns SubscriptionAttachFailed with the failing index, AlreadyExists and rollback failures. Work does not run, and no later plan entry is attempted.
- If acquisition raises, the same rollback runs for known successful acquisitions before the original exception is re-raised. The failing attach itself may have an uncertain outcome, which is the unresolved recovery boundary in Pending updates.
- After ordinary work return, every acquired cleanup is attempted. SubscriptionCompletion retains the work result beside failures listed in reverse acquisition order; a returned business Error does not trigger exceptional cleanup.
- AlreadyDetached means the native registration was absent when cleanup ran, including removal after handler failure. That absence meets the removal objective but remains a visible cleanup failure rather than certifying that the handler stayed attached during work.

BEHAVIOR

A refused acquisition skips the scoped work

Given an earlier subset of the plan attached successfully

When a later subscription's attach is refused

Then work is not invoked

Then earlier successful registrations receive cleanup attempts in reverse order

Then the result identifies the refused index and every failed rollback attempt

level: boundary area: Subscription scopes

BEHAVIOR

Normal completion retains independent cleanup failures

When scoped work returns normally

Then the returned work value is preserved

Then all known registrations receive reverse-order cleanup attempts

Then each failed cleanup remains indexed beside the work value

level: boundary area: Subscription scopes

3.6.2 Exceptional cleanup and resource limits

- The scope shim catches error, exit and throw with their stacktrace. Each detach runs independently so one detach exception does not skip later cleanup attempts.
- A work or acquisition exception's cleanup failures reach the configured reporter because there is no completion result to return. A reporter exception is caught and discarded before re-raising the original work exception; ADR-0005 records this outcome separation.
- Installation and removal are sequential rather than atomic. Concurrent emitters can see a partial registration set, and callbacks selected in another process can finish after scope exit.
- Captured resources must outlive independently executing callbacks, or applications must exclude concurrent emission during the scope. Scoped registration is suitable for explicit diagnostic/test lifetimes; it does not imply safe dynamic per-request subscriptions over a node-global event name.
- There is no work deadline, cleanup deadline or cancellation token in the scope API. A killed process or VM loss bypasses cleanup; normal native-call limits and user callback behavior still determine blocking.

BEHAVIOR

Cleanup reporting cannot replace a work exception

Given scoped work raises a catchable exception

When the scope attempts cleanup

Then every known registration receives a cleanup attempt

Then cleanup failures are offered to the configured reporter

Then the original exception class, reason and stacktrace are re-raised even if the reporter raises

level: boundary area: Subscription scopes

IMPLEMENTATION AND BEHAVIORAL EVIDENCE

Subscription acquisition, exception and cleanup shim, heterogeneous scope tests and scope and in-flight detach tests own the observable cases.

3.7 Correlation

A small shared value joins package-owned observations without owning their lifecycle.

Correlation contracts

responsibility

Admit a bounded observation value and expose interoperable optional or required metadata fields.

interface

`from_string`, `from_key`, `unique`, `to_string`, `field`, `required_field`, `max_bytes` and `describe_error`.

interactions

Applications select the unit of work; emitting packages propagate the value explicitly across helper processes and called packages.

invariants

The opaque value contains one to 128 UTF-8 bytes; optional and required field forms share one key and one present representation.

failure

Verbatim empty or oversized input is refused; foreign invalid values fail decoding; randomness or crypto runtime failures can raise.

3.7.1 Selection, validation and stable derivation

- `from_string` accepts one to 128 bytes verbatim and distinguishes empty from oversized input. The limit counts bytes rather than characters and therefore applies equally to multibyte external identifiers.
- `from_key` keeps a key that fits and hashes an empty or oversized key to 64 lowercase hexadecimal SHA-256 characters. The same key produces the same correlation across nodes and runs; every empty key therefore shares the same value.
- Stable hashing is suitable when application keys are used only for joining observations. A header or remote identifier that must remain verifiable verbatim uses `from_string`; hashing is not validation, authentication, secrecy or an injective identity protocol.

- `unique` draws 128 cryptographically random bits and emits 32 lowercase hexadecimal characters, redrawing an all-zero value. This has the shape of a W3C trace identifier and supplies probabilistic uniqueness rather than a coordinated global registry.
- Correlation is independent of package invocation ids and native SpanContext. Multiple calls of one business operation can share a correlation while keeping their own call/run/request identifiers; ADR-0007 records the selected scope.

BEHAVIOR

Stable key derivation preserves joins

When the caller derives a correlation from an application key

Then a key within the admitted byte range is preserved verbatim

Then an empty or oversized key produces its deterministic digest

Then repeated derivation of the same key produces the same value

level: boundary area: Correlation

3.7.2 Shared metadata contract and propagation

- The native key is the atom `correlation`; a present value is its UTF-8 binary. `field()` wraps required_field in a one-key optional codec, so None omits the key and foreign absent/nil/undefined values decode as None.
- `required_field()` decodes only a present binary within the admitted byte range. Optional and required readers interoperate for present values; a required reader rejects an absent value rather than inventing a correlation.
- A library generally accepts Option(Correlation) where work starts and copies it into every work-scoped occurrence and dependency call. Packages with an always-selected work value may use the required field; a Sinal observer must match the producer's possibility of absence.
- Start and stop span metadata are independent, so callers supply the correlation in both when terminal-only observers need it. Process-dictionary lookup and PID joins are not propagation contracts.
- Correlation has unbounded cardinality. Applications must not use it as a metric tag, and no Sinal limit bounds the number of distinct admitted values or records retained by an external collector.

BEHAVIOR

Required correlation readers reject absent values

Given a selected occurrence supplies no correlation

When a required correlation observer decodes the metadata

Then the typed callback is skipped with a missing-field failure

Then the registration remains when its failure observer returns normally

level: boundary area: Correlation

IMPLEMENTATION AND PUBLIC CONSUMER EVIDENCE

Correlation and byte bounds and field interoperability define the package boundary. The support_desk consumer carries a required ticket correlation across Fabric, Saga, HTTP Gun and LLM Wire; SSO Portal uses optional package values and labelled handlers rather than inferring identity from execution processes.

3.8 Native spans

Native telemetry owns timing while caller values keep separate event roles.

Span contracts

responsibility

Define and run typed start, stop and exception observations around one synchronous work call.

interface

`define`, `events`, `run`, Completion, role records, explicit time-unit readers and exception-term accessors.

interactions

Native telemetry span/3 supplies clocks, context, terminal choice and exception re-raise; the same Fields codecs describe caller data.

invariants

Protocol-owned keys cannot be overwritten; start and terminal share a context; ordinary work values produce stop rather than exception.

failure

Reserved-key definitions panic; malformed foreign timing fails decoding; catchable work/encoding exception emits exception and re-raises, while uncatchable termination can omit a terminal.

3.8.1 Event roles and owned fields

Role	Measurements	Metadata
Start	System time and monotonic time	Caller start metadata plus telemetry_span_context.
Stop	Duration, monotonic time and caller extra measurements	Caller stop metadata plus the same telemetry_span_context.
Exception	Duration and monotonic time	Caller start metadata plus context, kind, native reason and stacktrace.

- `events(span)` derives typed descriptors by appending start, stop and exception suffixes to one trusted name prefix. Each role can be observed independently, and native terms survive without JSON conversion.
- Extra stop measurements cannot declare duration or monotonic_time. Start metadata cannot declare telemetry_span_context, kind, reason or stacktrace, while stop metadata cannot declare telemetry_span_context.
- Start and stop caller metadata are independent. Stop does not inherit fields merely because they appeared at start; exception carries the start metadata with native exception details.
- Native context is opaque and supports equality. Context is shared within an invocation and distinct across invocations, while nested span contexts imply no parent-child trace relationship.
- SystemTime, MonotonicTime and NativeDuration wrap checked integer fields in separate types. Readers require an explicit TimeUnit; native conversion follows BEAM integer conversion semantics, and a foreign integer is not additionally constrained to a nonnegative duration by the decoder.
- ExceptionReason and ExceptionStacktrace retain arbitrary BEAM values behind opaque wrappers. Their explicit dynamic accessors are the escape boundary for applications that need native inspection.

3.8.2 Execution, terminal selection and failure timing

- `run` encodes start metadata before entering native span execution. A custom encoder exception at that point prevents start and work execution.
- Native execution records the initial monotonic value, emits start inline, invokes work and uses its Completion result to encode extra stop measurements and stop metadata. Timing measures the native span interval and can include inline instrumentation delay; it is not promised to measure only the body.
- A normally returned Completion emits stop and returns its result unchanged. That result may itself be Result.Error, which is a business value rather than a raised exception.
- A catchable work exception, or a catchable exception while encoding its completion, follows native exception emission using start metadata, then re-raises its original triple. Completion encoders are caller code; the total encoder signature does not make them non-raising.
- Spans bypass every route and have no deadline or cancellation setting. A slow start/stop/exception handler can delay span execution even when other occurrences are routed, and an untrappable termination can prevent terminal delivery.

BEHAVIOR

Normal business errors end a span with stop

Given work returns a completion whose business result is an error value

When the span completes normally

Then stop is emitted with the caller's independent stop metadata

Then the error value is returned unchanged

Then no exception occurrence is emitted for that returned value

level: boundary area: Native spans

BEHAVIOR

A catchable work exception retains native evidence

When instrumented work raises a catchable error, exit or throw

Then the exception occurrence shares the start context and carries its class, reason and stacktrace

Then the original exception is re-raised

Then stop is not emitted for that invocation

level: boundary area: Native spans

- Nested normal execution observes outer start, inner start, inner terminal and outer terminal. Handler order within any one occurrence remains unspecified, and separate processes need no global span order.

IMPLEMENTATION AND BEHAVIORAL EVIDENCE

Typed native spans, native span adapter, terminal, context and native-term tests and explicit unit and malformed timing tests retain the native semantics.

CONTEXT

Verification

4 Verification

4.1 Foundation

Native parity, public consumers and synchronized probes verify each contract at its real boundary.

GOAL

Keep native behavior and typed boundaries executable

Every contract in the observation and forwarding contexts is checked by a test, a native harness or a consumer. A claim without an executable check is marked as convention.

GOAL

Keep documentation examples executable

Usage is documented as compiled, runnable examples (each a Usage example) in a separate package that depends on this one, and README snippets occur verbatim in them. Public module introductions are checked mechanically. A documented example cannot drift from compiling source, and no prose usage guide stands beside the examples.

NO-GOAL

Measurements are not performance guarantees

Benchmark and stress output is observational evidence from one workload and one machine. It establishes no latency bound, throughput promise or capacity limit.

INVARIANT

A validation profile runs one nonempty successful test summary

The gate accepts exactly one successful Gleeunit summary with a nonzero test count. Empty, missing, failed, skipped and ambiguous summaries fail the profile.

mechanism

INVARIANT

Race evidence uses barriers, not elapsed time

A synchronization barrier establishes that a competing process reached a causal step. A sleep can bound a wait but never proves ordering.

mechanism

PRINCIPLE

Evidence states its own limits

Each check names what it proves and what it leaves open. A passing syntax gate does not prove semantic fidelity, and a passing oracle mapping does not prove every upstream case.

4.2 Validation profiles

One gate script owns the checks that local runs and CI share.

- The gate script offers three profiles: `fast`, `full` and `benchmark`. `fast` checks full-tree formatting, authored tooling lint, Gleam formatting and build, independent native compilation, gate controls, the package test suite and the usage examples.
- `full` cleans the build and adds the focused native behavior program, the stress harness, every synchronized forwarder race and the design gate. `benchmark` compiles Gleam and authored Erlang, runs the benchmark program and retains its raw output, source SHA-256 hashes and command metadata.
- Native compilation uses `erlc -Werror` for authored production and test Erlang and for the race-probe asset, with dependency include paths and generated BEAM code paths. Gleam's `--warnings-as-errors` does not promote native Erlang warnings, so this check is separate.
- Gate controls compile a valid module and require an unused-variable module to fail with its intended diagnostic. Further controls reject empty, missing, failed, skipped and ambiguous test summaries.

- `nix flake check` verifies the original tree without formatting it. The flake pins actionlint, ShellCheck and Ruff; actionlint invokes ShellCheck for workflow shell commands, and ShellCheck also checks `.envrc`.
- Ruff checks authored `dev/*.py` for import, syntax and undefined or unused names (E4/E7/E9/F), and treefmt applies Ruff formatting to the same scope. Formatting excludes build output, rendered documents, transient artifacts and frozen benchmark provenance.
- The gate retains each check's raw log and a `summary.json` with command, exit code, elapsed time and successful test count. A failed or timed-out command fails the profile and leaves the collected evidence in place.
- The CI workflow runs `full` on every push, pull request and manual dispatch. Its final status job accepts only a successful validation job, so failure, cancellation and skipping cannot pass. Validation artifacts are retained for 14 days, including failed runs, and missing expected evidence fails collection.
- A separate workflow runs `benchmark` weekly and on manual dispatch and retains its artifacts for 30 days. Its result is observational, carries no latency threshold and is independent of the required CI status.
- The Nix development shell pins Gleam, Erlang/OTP 29, rebar3 and Python. CI runs the same gate through that shell, so the supported toolchain is the flake lock rather than a separate CI matrix. ADR-0010 records the profile split.

Obligation	Check	Profile	Enforcement
Canonical formatting and tooling correctness	`nix flake check --print-build-logs`; `gleam format --check src test examples/usage/src examples/usage/test`	fast, full	mechanism
Warning-free authored build	`gleam build --warnings-as-errors`; the gate's `erlc -Werror` command	fast, full, benchmark	mechanism
Gate refusal and nonempty test execution	`python3 dev/test_gate.py`; `gleam test`	fast, full	mechanism
Usage examples compile warning-free and run	`gleam build --warnings-as-errors` and `gleam test` in `examples/usage`	fast, full	mechanism
Native dispatch, concurrency and incarnation races	`gleam run -m focused_behavior`; `gleam run -m stress_test`; `python3 dev/check_forwarder.py`	full	mechanism for the bounded scenarios
Design artifact integrity	`nix run .#design-gate-check -- docs/design .`	full	mechanism for artifact and reference checks
Workload-dependent performance observation	`gleam run -m benchmark`	benchmark	partial; no universal latency claim
Semantic design fidelity and separate consumer use	Design review and consumer packages	on boundary changes	convention

4.3 Native oracle

Telemetry 1.4.2 is the differential oracle for Sinal's native boundary.

4.3.1 Pinned source and mapping scope

- The oracle is Native oracle: telemetry 1.4.2 at upstream revision `7baf8085e406d5ae9e43b284d7c866742ae04b28` under Apache 2.0, with Hex tarball checksum `928F6495066506077862C0D1646609EED891A4326BEE3126BA54B60AF61FEBB1`. The package manifest resolves 1.4.2 while the public dependency range admits later 1.x releases.
- Sinal replaces dynamic application terms with typed `Event` and `Fields` and preserves native atom-list names, measurement and metadata maps, references and exceptions. The wrapper converts none of those values through JSON.
- The mapping states semantic checks rather than a literal port of every upstream assertion. It does not claim to run every check in both upstream registry groups, and it adds Sinal-specific checks for typed decoding, scoped acquisition, correlation and forwarding.
- A fresh oracle harvest must record its own checkout, dependencies and result; a mapping change is not a harvest. ADR-0009 records the pin.

4.3.2 Case mapping and dispositions

- Every upstream case receives one disposition: adopted, adapted, no typed facade, adapter-specific, excluded or not claimed. The case-by-case table, its evidence and the normalization of each case live in the oracle reference, which also pins the sources and says how to rerun the comparison.
- Malformed application-map removal is adapted. Sinal reports a malformed measurement or metadata failure, skips that one invocation and keeps the registration, as ADR-0004 records.
- Native introspection, the legacy two-argument entry point and the upstream test-helper API have no typed facade, because explicit codecs and Gleam assertions replace them. Raw callers and tests still use native introspection.
- Running every wrapper check under both the `ets` and `persisted` registry groups is not claimed, because it needs its own executable matrix. Sinal tests the raw `persist/0` transition separately.
- The local benchmark workloads and their retained run are in the benchmark reference, which carries no behavioral claim.

4.4 Focused checks and consumer obligations

Native harnesses and public consumers cover what unit tests cannot.

Verification contracts

responsibility

Keep native behavior, typed boundaries, race guarantees and documentation examples executable.

interface

Package tests, the focused behavior and stress programs, the peer-VM startup check, the forwarder race harness, the formatter and the design gate.

interactions

Native telemetry is the differential oracle; consumer packages exercise public imports and real application types.

invariants

Checks use the declared toolchain; barrier synchronization governs race evidence; README snippets match their executable source exactly.

failure

A behavior mismatch, stale example, missing public module documentation, race-probe failure or stale design render fails its owning check.

4.4.1 Native and race harnesses

- The focused program verifies native attachment, decoded values and emitter-process identity. The stress harness exercises bounded attach and detach churn, concurrent emissions and span-context isolation.
- The forwarder checker copies production source into a temporary directory and adds synchronization barriers there, leaving package source unchanged. Its scenarios select publication, admission-restart and drop-notice races, and an option selects a candidate source tree.
- The checker fails when its production anchors drift. An anchor failure requires updating the instrumentation and its race assertions together and is not evidence of a runtime defect.
- Native callback placement, in-flight detach and restart guarantees use process barriers. ADR-0008 records why admission authority is incarnation-local.

4.4.2 Consumers, examples and documentation

- A separate consumer package must use public imports to verify the ordinary path, advanced configuration, caller-owned records and typed failure handling. Package-internal tests establish native behavior, and consumer compilation establishes what an external caller can use.
- The usage examples are a package of their own under `examples/usage`, one module per topic, each with a `run` function that asserts what the example does. The validation profiles build them without warnings and run them after the package tests. The README test extracts every Gleam fence from the README and requires its exact text to occur in one of those modules, so a README example changes together with its executable source. ADR-0014 records why usage lives in examples and why the oracle and benchmark references stay as linked documents.
- A module-documentation test requires an introduction for every public Gleam module, and internal modules stay under the declared internal-module pattern.

- A custom codec needs exact-key encoding, round-trip and malformed-foreign-value checks. An enumeration needs a test that lists every constructor and round-trips each one, because an exhaustive naming function does not prove that the value list is complete.
- Documentation generation and a local hex tarball inspection check the public modules, native Erlang files, license, README and dependency manifest before an authorized release. Publishing and version decisions are separate actions.

4.5 Benchmark method

A benchmark run is a frozen, provenance-tagged observation.

- The benchmark program reports runtime identity, warmup and timed iteration counts, total elapsed time, average time and throughput for zero-handler emission, typed synchronous handling, codec work and native spans. Route lookup reports averages with no routes and with one unrelated route.
- Each workload is one timed loop whose average is total monotonic nanoseconds divided by iterations using integer division, including loop overhead. The result is a loop average, not a distribution of per-operation samples or a percentile.
- A recorded run keeps its workload parameters, runtime versions, source SHA-256 hashes and raw output together. A comparison with another run requires the same parameters and a stated workload.
- The result establishes no production latency, handler-time bound, equality with raw telemetry, forwarder throughput or capacity in bytes. ADR-0011 records the accepted measurements and their limits.
- How to run the program, the environment and the retained run are in the benchmark reference, and the pinned native reference is in the oracle reference. Neither states a guarantee.

Glossary

Every term the layer declares, with the context that owns it.

Admission target

owned by forwarding

The direct event destination, capacity and fresh admission counters that one incarnation publishes together. A producer that read an old target can reach only that incarnation.

Attachment

owned by observation

A handle naming one native handler registration. A copied handle retains the same identifier after that registration disappears.

Benchmark run

owned by verification

One execution of the benchmark program with fixed workloads and iteration counts, retained with its runtime identity and source hashes. It is an observation, not a guarantee.

Cleanup failure

owned by observation

A failed detach attempt during scope cleanup, retained with its zero-based plan index. It never replaces the work value or the work exception.

Correlation

owned by observation

An opaque UTF-8 value of one to 128 bytes identifying an application-selected unit of work across observations. It is distinct from a package's own invocation identifier.

Drop report

owned by forwarding

A best-effort observation of capacity refusals, unavailable destinations, and a restart's estimate of lost in-flight events. It is a diagnostic count rather than a delivery receipt.

Event

owned by observation

An immutable typed definition of one telemetry name and its measurement and metadata shapes. A definition is distinct from an occurrence emitted under that name.

Evidence log

owned by verification

The raw output, command, exit code, elapsed time and test count that a check retains. A failed check leaves its log in place.

Fields

owned by observation

A bidirectional description of the declared keys in one native telemetry map and the caller value represented by those keys.

Forwarder

owned by forwarding

An immutable configuration for a named, capacity-bounded telemetry execution process. Its running process lifetime is a forwarder incarnation.

Forwarder incarnation

owned by forwarding

One running lifetime of a forwarder process, with its own event destination and admission counters.

Handler failure

owned by observation

A malformed measurement, malformed metadata, or caller-defined error returned by a typed handler. Unexpected exceptions are outside this sum.

Handler identifier

owned by observation

The node-local native identity of one registration. It is either a fresh generated value or a nonempty stable string chosen by the caller.

In-flight event

owned by forwarding

A forwarded occurrence admitted to one incarnation whose dispatch has not finished. Capacity includes both queued and executing occurrences.

Measurements

owned by observation

The numeric or application-defined values recorded as the first map of a telemetry occurrence.

Metadata

owned by observation

The descriptive and identifying values recorded as the second map of a telemetry occurrence.

Native oracle

owned by verification

The pinned upstream telemetry release against which Sinal's native boundary is compared. It supplies expected behavior for mapped cases and no claim beyond them.

Native registry

owned by observation

The node-wide `:telemetry` handler table that selects and runs registered callbacks. Sinal installs typed callbacks into it and creates no registry of its own.

Race probe

owned by verification

A check that runs a temporary copy of production source with synchronization barriers added. It establishes causal ordering of competing processes without changing package source.

Refusal

owned by forwarding

The immediate typed rejection of an occurrence because the destination is full or unpublished. A refusal is neither retried nor delivered inline.

Route

owned by forwarding

A node-wide binding from an event-name prefix to a forwarder. The most specific matching prefix selects the destination.

Source definition

owned by observation

An event name, route prefix or field key written in application source and validated against the name grammar. Source definitions are the only input that may create atoms.

Avoid: dynamic name — a name built from input creates unbounded permanent atoms.

Span

owned by observation

An immutable definition of the start, stop, and exception observations surrounding one call of instrumented work.

Span completion

owned by observation

The normal result of instrumented work together with its extra stop measurements and independent stop metadata. A returned application error is a normal completion.

Span context

owned by observation

A native identity shared by the start and terminal observations of one span invocation. It supplies equality rather than trace parentage.

Subscription

owned by observation

An immutable registration description whose typed callback, failure observer, and selected event definitions are already bound.

Subscription plan

owned by observation

An ordered group of subscriptions with a reporter for cleanup failures that accompany a catchable exception.

Subscription scope

owned by observation

One acquisition, work, and cleanup lifetime for a subscription plan. It owns the registrations it successfully acquired.

Usage example

owned by verification

A compiled, runnable module under `examples/usage` that shows one public-API topic and asserts its result. The README quotes its code verbatim, and the validation profiles build and run it.

Validation profile

owned by verification

A named set of checks (`fast`, `full` or `benchmark`) that the gate script runs identically for local use and CI.